Ad Soyad: _____	Tarih: _____
Proje Ödevi – SPORSIS – *Spor Salonu Üyelik Sistemi*	

Proje Hakkında

Bu projede bir **Spor Salonu Üyelik Sistemi (SPORSIS)** geliştireceksiniz. Sistem; üye yönetimi, üyelik paketleri, üyelik kayıtları ve antrenman takibi işlevlerini içerecektir. Proje dönem boyunca 4 fazda geliştirilecek ve her faz bir öncekinin üzerine inşa edilecektir.

- **Faz 1 (Hafta 1–3):** STL kapsayıcıları ve modern C++ ile veri katmanı
- **Faz 2 (Hafta 4–8):** Qt6 ile grafiksel kullanıcı arayüzü
- **Faz 3 (Hafta 9–11):** Çok kanallı programlama ile performans iyileştirmesi
- **Faz 4 (Hafta 12–14):** Ağ programlama ile istemci/sunucu mimarisi

Süre: 14 hafta (dönem boyunca) **Başlangıç kodu:** `out/Projects/SPORSIS/`

Teknoloji Gereksinimleri

Bu proje **C++20** standardı ve **Qt6** çerçevesi kullanılarak geliştirilmelidir. Aşağıdaki araçların sisteminizde kurulu olması gerekir:

- **Derleyici:** g++ veya clang++ (C++20 desteği ile)
- **Qt6:** Qt 6.5+ (Widgets, PrintSupport, Charts modülleri)
- **CMake:** 3.20 veya üzeri

Faz 1 derleme:

```
g++ -std=c++20 -o sporsis src/main.cpp
```

Faz 2+ derleme (Qt ile):

```
mkdir build && cd build
cmake .. -DCMAKE_PREFIX_PATH=/path/to/Qt/6.x
cmake --build .
```

1 Alan Modeli

Sisteminiz aşağıdaki temel varlıkları içerecektir:

Varlık	Anahtar	Alanlar
`Uye`	`uye_no`	isim, soyisim, telefon, kayıt tarihi
`UyelikPaketi`	`paket_id`	paket adı, süre (gün), fiyat
`UyelikKaydi`	`kayit_id`	üye no, paket id, başlangıç, bitiş, durum
`AntrenmanKaydi`	`kayit_id`	üye no, tarih, süre (dk), aktivite

Durum Enum'ları

```
enum class UyelikDurum {
    Aktif,              // Üyelik hala geçerli
    SuresiDolmus,       // Üyelik süresi sona ermiş
    Dondurulmus         // Üyelik geçici olarak dondurulmuş
};
```

Örnek Struct Tanımı

```
struct Uye {
    int uye_no{};
    std::string isim;
    std::string soyisim;
    std::string telefon;
    std::string kayit_tarihi;  // "YYYY-MM-DD" formatında
};
```

Varlık İlişkileri

- Bir `Uye` birden fazla `UyelikKaydi`'na sahip olabilir (1:N)
- Bir `UyelikPaketi` birden fazla `UyelikKaydi`'nda yer alabilir (1:N)
- Bir `Uye` birden fazla `AntrenmanKaydi`'na sahip olabilir (1:N)
- `UyelikKaydi`, `Uye` ve `UyelikPaketi` arasında köprü görevi görür

2 Faz 1: STL ve Veri Katmanı (Hafta 1–3)

Faz 1 Hedefi

Bu fazda projenin veri katmanını oluşturacaksınız. Tüm veri yapıları STL kapsayıcıları ile yönetilecek, dosya okuma/yazma işlemleri ikili (binary) formatta gerçekleştirilecektir. **Bu fazda henüz Qt kullanılmayacaktır;** yalnızca C++20 standart kütüphanesi kullanılacaktır.

Görev 1.1 – Varlık Struct'ları

`include/varliklar.h` dosyasında aşağıdaki struct'ları tanımlayın:

- `Uye`, `UyelikPaketi`, `UyelikKaydi`, `AntrenmanKaydi` struct'ları
- `UyelikDurum` enum class'ı
- Her struct için `operator«` (akış çıkış operatörü)
- En az bir alanda `std::optional` kullanımı (örneğin bitiş tarihi)

Görev 1.2 – Depo Şablon Sınıfı

`include/depo.h` dosyasındaki `Depo<K,V>` şablon sınıfını tamamlayın:

- `ekle(anahtar, deger)`: Yeni kayıt ekler
- `bul(anahtar) -> std::optional<V>`: Anahtar ile arama yapar
- `sil(anahtar) -> bool`: Kaydı siler
- `tumunu_al() -> const std::map<K,V>&`: Tüm kayıtları döner
- `filtrele(kosul) -> std::vector<V>`: Lambda ile filtreleme

Görev 1.3 – STL Kapsayıcı Kullanımı

Aşağıdaki senaryolarda uygun STL kapsayıcılarını kullanarak çözümler yazın:

- `std::unordered_map` ile üye kart numarasından hızlı üye erişimi
- `std::deque` ile antrenman geçmişi (baştan ve sondan ekleme/çıkarma)
- `std::set` ile saat dilimi çakışma kontrolü (sıralı benzersiz slotlar)
- Aktif üyelikleri filtreleme (C++20 ranges)
- En çok antrenman yapan üyeyi bulma

Görev 1.4 – Dosya İşlemleri

Verileri ikili dosyada saklayın ve okuyun:

- `dosyaya_kaydet(dosya_yolu, depo)`: Depo içeriğini dosyaya yazar
- `dosyadan_oku(dosya_yolu) -> Depo`: Dosyadan veri okur
- Program kapanırken otomatik kaydetme, açılırken otomatik yükleme

İpucu: Ranges Pipeline

C++20 ranges ile sorgu yazma örneği:

```
auto aktifler = kayitlar
    | std::views::filter([](const auto& k) {
          return k.durum == UyelikDurum::Aktif;
      })
    | std::views::transform([](const auto& k) {
          return k.uye_no;
      });
```

3 Faz 2: Qt6 ile Grafiksel Arayüz (Hafta 4–8)

Faz 2 Hedefi

Bu fazda Faz 1'de oluşturduğunuz veri katmanının üzerine Qt6 ile bir masaüstü arayüzü inşa edeceksiniz. **Qt6 Widgets** modülü kullanılacak ve **CMake** ile derlenecektir.

Görev 2.1 – Haftalık Ders Programı Tablosu

`QTableWidget` veya `QAbstractTableModel` ile haftalık grup dersi programı oluşturun:

- Satırlar: Saat dilimleri (09:00–10:00, 10:00–11:00, ...)
- Sütunlar: Haftanın günleri (Pazartesi–Pazar)
- Hücrelerde ders adı, eğitmen ve kapasite bilgisi
- Dolu/boş slot renklendirmesi

Görev 2.2 – Üye Giriş/Çıkış Kaydı

Üyelerin salona giriş ve çıkış zamanlarını takip eden ekran:

- Üye kart numarası girişi (`QLineEdit` + barkod simülasyonu)
- Giriş/çıkış zaman damgası (`QDateTime`)
- Günlük giriş listesi (`QTableView`)
- Anlık salon doluluk göstergesi (`QProgressBar`)

Görev 2.3 – Eğitmen Atama ve Üyelik Yenileme

- Eğitmen atama ekranı: Üye seçimi, eğitmen seçimi (`QComboBox`), program oluşturma
- Üyelik yenileme dialogu (`QDialog`): Mevcut paket bilgisi, yeni paket seçimi, ödeme onayı, başlangıç/bitiş tarihi hesaplama (`QDateEdit`)
- Form doğrulama: Boş alan kontrolü, telefon formatı kontrolü

Görev 2.4 – Raporlama ve Grafik

- Aylık gelir grafiği (`QChart` – `QBarSeries`): Paket türüne göre gelir dağılımı
- Pencere boyutu ve konumunu `QSettings` ile kaydetme/yükleme
- JSON formatında dışa aktarım (`QJsonDocument`)

İpucu: Sinyal/Slot Bağlantısı

Qt6'da yeni bağlantı söz dizimini kullanın (derleme zamanı kontrolü):

```
connect(girisButon, &QPushButton::clicked,
        this, &MainWindow::uyeGirisKaydet);
```

4 Faz 3: Çok Kanallı Programlama (Hafta 9–11)

Faz 3 Hedefi

Bu fazda uygulamanızı çok kanallı hale getirerek uzun süren işlemlerin arayüzü dondurmamasını sağlayacaksınız. `std::thread`, `std::mutex` ve Qt'nin `QThread` mekanizması kullanılacaktır.

Görev 3.1 – Turnike Giriş Simülasyonu

Eş zamanlı (concurrent) turnike giriş senaryosunu simüle edin:

- Birden fazla turnike izleği (thread) aynı anda üye girişi yapar
- Her turnike kart okuma, yetki kontrolü ve kapı açma adımlarını gerçekleştirir
- Paylaşılan salon kapasite sayacı thread-safe olmalı
- Salon dolu ise giriş reddedilmeli (kapasite kontrolü)

Görev 3.2 – Arka Plan Kapasite İzleme

- Arka planda çalışan izlek salon kapasitesini periyodik olarak kontrol eder
- Kapasite %90 üzerine çıktığında GUI'ye uyarı sinyali gönderir
- Paylaşılan veri yapılarını `std::mutex` / `std::lock_guard` ile koruma
- İptal mekanizması: Kullanıcı izleme görevini durdurabilmeli

İpucu: QThread ile İlerleme Bildirimi

```
class KapasiteIzleyici : public QObject {
    Q_OBJECT
signals:
    void kapasiteGuncellendi(int mevcutKisi);
    void kapasiteUyarisi(const QString& mesaj);
    void izlemeDurduruldu();
};
```

5 Faz 4: Ağ Programlama (Hafta 12–14)

Faz 4 Hedefi

Bu fazda uygulamanızı istemci/sunucu mimarisine dönüştüreceksiniz. Merkezi bir sunucu tüm verileri yönetecek, resepsiyon terminali ve eğitmen ekranı birer istemci olarak bağlanacaktır. **Qt Network** modülü ve **TCP soketleri** kullanılacaktır.

Görev 4.1 – Merkezi Sunucu

`QTcpServer` ile merkezi sunucu oluşturun:

- Birden fazla istemciyi eş zamanlı kabul etme
- JSON tabanlı mesaj protokolü (istek/yanıt formatı)
- Üyelik durumu sorgulama ve giriş yetki kontrolü

Görev 4.2 – Resepsiyon Terminali ve Eğitmen Ekranı

- Resepsiyon terminali (`QTcpSocket`): Üye kayıt, üyelik yenileme, giriş/çıkış
- Eğitmen ekranı (ayrı istemci): Günlük program görüntüleme, üye antrenman geçmişi
- Bağlantı kopması durumunda yeniden bağlanma denemesi

İpucu: JSON Mesaj Protokolü

```
// Istek formati
{
    "tip": "giris_kontrol",
    "uye_no": 2001,
    "turnike_id": 1
}

// Yanit formati
{
    "durum": "basarili",
    "mesaj": "Giris onaylandi",
    "uyelik_bitis": "2026-09-15"
}
```

6 Değerlendirme Kriterleri

Faz	Puan	Değerlendirme Alanları
Faz 1: STL ve Veri Katmanı	25	Struct tasarımı, Depo şablonu, STL kapsayıcıları (unordered_map, deque, set), dosya I/O, `optional` kullanımı
Faz 2: Qt GUI	35	Ders programı tablosu, giriş/çıkış kaydı, eğitmen atama, üyelik yenileme, QChart, kod kalitesi
Faz 3: Çok Kanallı	20	Turnike simülasyonu, kapasite izleme, mutex/lock_guard, iptal mekanizması
Faz 4: Ağ Programlama	20	TCP sunucu/istemci, JSON protokol, resepsiyon terminali, eğitmen ekranı, hata yönetimi
Toplam	**100**	

7 Teslim Edilecekler

Her faz sonunda aşağıdaki dosyaları teslim edin:

Faz 1 Teslimi (Hafta 3 sonu)

- `include/varliklar.h` – Varlık struct'ları ve enum'lar
- `include/depo.h` – Depo şablon sınıfı
- `src/main.cpp` – Test ve demonstrasyon kodu
- `CMakeLists.txt` – Derleme yapılandırması

Faz 2 Teslimi (Hafta 8 sonu)

- Faz 1 dosyaları + Qt GUI kaynak dosyaları
- `src/main_window.h/cpp` – Ana pencere
- `src/uyelik_model.h/cpp` – Üyelik tablo modeli
- Güncellenmiş `CMakeLists.txt` (Qt6 bağımlılıkları ile)

Faz 3 Teslimi (Hafta 11 sonu)

- Faz 2 dosyaları + thread sınıfları
- Turnike simülasyonu ve kapasite izleme uygulaması

Faz 4 Teslimi (Hafta 14 – Final)

- Tüm kaynak kodlar
- Sunucu, resepsiyon terminali ve eğitmen ekranı uygulamaları
- Kısa kullanım kılavuzu (README.md)
- Proje sunumu (5–10 dakika)